

MoE Serving 毕设选题方案

Idea A: 面向通信高效 MoE 推理的 Gate 感知误差有界 Partial Combine

Idea B: 能效/SLO 双约束下的 MoE 推理联合优化

2026 年 6 月 4 日

面向通信高效 MoE 推理的 Gate 感知误差有界 Partial Combine

定位

MoE inference 的主要瓶颈是 expert-parallel all-to-all communication。Forward pass 中 dispatch 与 combine 构成两次对称 all-to-all，其中 combine 目前通常被视为必须完整传输的 weighted aggregation step。是否可以利用 gate-weighted linear combine 的结构，在 hidden-state perturbation 有界下减少第二次 all-to-all 的昂贵链路流量。

Gate-aware error-bounded partial combine

combine 本质

$$y_t = \sum_{e \in S(t)} g_{t,e} \cdot o_{t,e},$$

线性可结合；且 gate weight $g_{t,e}$ 在 combine 之前就已知，天然量化了每个 expert 的贡献大小。于是可在跨昂贵链路前丢弃/粗传 gate 权重极小的 expert 贡献，误差可解析界定：

$$\|\Delta y_t\| \leq \sum_{e \in \text{dropped}} g_{t,e} \cdot \|o_{t,e}\|$$

方案 A

研究问题

1. 给定误差 ϵ , partial combine 能省多少昂贵链路流量? 指标: expensive-tier bytes、P99 TBT、accuracy drop, 画 accuracy-traffic 帕累托前沿。
2. co-activation + importance-aware placement 能否进一步放大 partial combine 收益? 把 placement 从普通 topology-aware 升级为重要性加权:

$$\min \sum_{i,j} C_{ij} \cdot l_{ij} \cdot \text{cost}(loc_i, loc_j)$$

其中 C_{ij} 是 expert i, j 的 co-activation 频次, l_{ij} 是由 gate weight / contribution 估计的重要性 $\rightarrow$ 把大贡献对收进便宜 locality 精确预聚合, 小贡献对放远端可丢/粗传。

- ▶ co-activation: 多个 expert 被同一个 token 同时选中的频率。频率高 $\rightarrow$ 放同一个便宜 locality 就有更多预聚合机会。
- ▶ importance-aware: locality 显存/容量有限, 不能把所有高频共现对都塞一起。 l_{ij} 排优先级——gate 大、贡献高的 expert 对冲在前面, 因为它们每一次预聚合省下的字节最多; gate 小的对即使共现频繁, 省下的也有限, 往后排。

必要的前置实验

gate weight 能否体现 expert output contribution? 若 $g_{t,e}$ 小, $g_{t,e} o_{t,e}$ 对最终 hidden state 的影响是否真的小? 画出丢弃尾部 gate 后的 hidden-state 偏差与下游精度退化曲线。

方案 A

大概建模的方向

- ▶ 决策变量：每个 token 哪些 expert 贡献被丢弃/近似传输，受 ϵ 预算约束；expert $\rightarrow$ locality 放置 x ，重要性加权图分割；partial combine 在哪一层 locality 汇合。
- ▶ 目标：min (expensive-tier bytes / P99 TBT), s.t. 每 token 误差 $\leq \epsilon$ 。
- ▶ 约束：误差预算 ϵ + 昂贵链路带宽（层次化、异构）+ 显存/容量 + TTFT/TBT SLO。
- ▶ 方法：误差有界下的 MILP 精确解（小规模）+ 重要性加权 co-activation 图分割 / 启发式（大规模）。

一些想到的内容

- ▶ 拓扑：单集群内，网络越往上越挤。GPU 之间走 NVLink 几乎不拥塞 (1:1)，节点到机架交换机稍挤 ($\sim 1:1$)，机架到 pod 交换机更挤 ($\sim 4:1$)，跨 pod 经 spine 最挤 ($\sim 8:1$)。
- ▶ 模型/负载：DeepSeek-V2-Lite，优先大 top-k 的模型，co-activation 机会多。
- ▶ Baselines：完整 combine（无丢弃）、无重要性的 topology-aware placement、RailS 思路的纯拓扑选路。
- ▶ Metrics：expensive-tier bytes、P99 TBT、accuracy drop、accuracy-traffic 帕累托前沿。

能效/SLO 双约束下的 MoE 推理联合优化

现有 MoE 推理系统全在优化 latency/throughput, 可以建一个 MoE 专属能耗模型 (静态 + 动态计算 + 通信三部分), 在 SLO 约束下, 联合优化 expert 放置 / 副本数 / 功率感知 batching, 最小化 J/token (并在功率预算 P_{cap} 下最大化吞吐)。

核心: MoE 比 dense 多出三个 expert-level 能耗杠杆

Dense 能耗工作的杠杆只有 (实例数 / 并行度 / GPU 频率); MoE 额外有三个特别之处:

1. expert 放置 → 决定 all-to-all 通信能耗;
2. expert 副本数 → 用“多副本占更多卡的静态功耗”换“更少通信 / 更均衡负载的能耗”;
3. 路由依赖的瞬时功率波动 → MoE 稀疏激活使各 GPU 功率天然不均、有峰值结构。这给了一个 dense 没有的在线 knob: 功率感知 batching。

三个判断

- ▶ 静态功耗: expert 全得存在某张 GPU 上, 放置越分散、副本越多 → 点亮的 GPU 越多 → 静态功耗越大。
- ▶ 通信功耗: MoE 计算稀疏但 all-to-all 密集, 通信能耗是 MoE 独有大头。
- ▶ latency-最优 $\neq$ energy-最优: 延迟最低的配置和能耗最低的配置通常不是同一套。必须用实验证明这一点 → 画 latency-energy 散点图。

方案 B

研究问题

1. 建 MoE 推理能耗模型（静态/动态计算/通信三项），实测 latency-optimal vs energy-optimal 配置差距。
2. energy-aware expert replication & placement——SLO 约束下决定每个 expert 放哪、留几个热副本，最小化 J/token。最干净、最 MoE，建议主攻。
3. 在功率预算 P_{cap} 约束下做功率感知 token-to-batch 分组——利用 MoE 路由确定性、功率需求可提前预知，削峰填谷，在 $\forall g, t: P_{g,t} \leq P_{cap}$ 下最大化吞吐。

大概建模的方向

能耗（每 decode step / batch）：

$$E_{\text{total}} = \underbrace{\sum_g P_g^{\text{stat}} \cdot T}_{\text{静态 } (\propto \text{点亮 GPU 数})} + \underbrace{\sum_e C_e^{\text{cmp}} \cdot a_e}_{\text{动态计算 } (\propto \text{激活 token})} + \underbrace{\sum_{(i,j)} C_{ij}^{\text{comm}} \cdot D_{ij}}_{\text{all-to-all 通信}}$$
$$\text{J/token} = E_{\text{total}} / \text{tokens}$$

方案 B

大概建模的方向

- ▶ 决策变量: expert $\rightarrow$ GPU 放置 x_{eg} ; 每个 expert 热副本数 r_e ; 功率感知 batch/并发策略 (可选 DVFS 频率 f_g)。
- ▶ 目标: $\min J/\text{token}$ 。
- ▶ 约束: TTFT/TBT SLO + 显存容量 + (RQ3) 瞬时功率 $\forall g, t: P_{g,t} \leq P_{cap}$ 。
- ▶ 方法: 小规模 MILP 求可证明最优 + 大规模拉格朗日松弛/启发式 + (可选) 轻量负载预测器提前调副本。

一些想到的内容

- ▶ 现成数据: TokenPowerBench (开源, J/token, GPU/节点/系统三级)、HotCarbon'25 「Energy Efficient or Exhaustive?」 (功率数学模型 + 实测分解)、JouleBench、Microsoft 「Energy Use of AI Inference」 (bottom-up 每查询能耗)。
- ▶ Metrics: J/token、SLO 命中率、能耗 vs latency-only 的节省、帕累托前沿。